# Task 4

```
create database orders;
use orders;
create table customer (
    customer_id int primary key auto_increment,
    customer_name varchar(100) not null,
    email varchar(100) not null unique
);
create table product (
    product_id int primary key auto_increment,
    product_name varchar(100) not null,
    price decimal(10,2) not null check (price > 0)
);
create table orders (
    order_id int primary key auto_increment,
    customer_id int not null,
    order_date date not null,
    total_amount decimal(10,2) not null default 0
        check (total_amount >= 0),
    order_status varchar(20) not null default 'pending'
        check (order_status in
        ('pending', 'shipped', 'delivered', 'cancelled')),
    foreign key (customer_id)
        references customer(customer_id)
);
create table order_details (
    order_detail_id int primary key auto_increment,
    order_id int not null,
    product_id int not null,
    quantity int not null check (quantity > 0),
```

```
    price decimal(10,2) not null check (price > 0),
    unique (order_id, product_id),
    foreign key (order_id)
        references orders(order_id) on delete cascade,
    foreign key (product_id)
        references product(product_id)
);
insert into customer (customer_name, email) values
('arun', 'arun@gmail.com'),
('priya', 'priya@gmail.com'),
('ravi', 'ravi@gmail.com'),
('divya', 'divya@gmail.com'),
('kumar', 'kumar@gmail.com');

insert into product (product_name, price) values
('laptop', 50000),
('mouse', 500),
('keyboard', 1000),
('monitor', 12000),
('headphones', 2000),
('printer', 8000);

select
    c.customer_id,
    c.customer_name,
    o.order_id,
    o.order_date,
    o.total_amount,
    o.order_status
from customer c
join orders o on c.customer_id = o.customer_id
```

```
order by c.customer_id, o.order_date;


select
    p.product_id,
    p.product_name,
    o.order_id,
    c.customer_name,
    od.quantity,
    od.price,
    od.quantity * od.price as item_total
from product p
join order_details od on p.product_id = od.product_id
join orders o on od.order_id = o.order_id
join customer c on o.customer_id = c.customer_id
where o.order_status <> 'cancelled'
order by p.product_name;


select
    p.product_id,
    p.product_name,
    count(od.order_detail_id) as times_ordered
from product p
left join order_details od on p.product_id = od.product_id
left join orders o on od.order_id = o.order_id
    and o.order_status <> 'cancelled'
group by p.product_id, p.product_name;


select
    p.product_name,
    coalesce(sum(
        case
```

```
            when o.order_status <> 'cancelled'
            then od.quantity
            else 0
        end
    ), 0) as total_quantity_sold
from product p
left join order_details od on p.product_id = od.product_id
left join orders o on od.order_id = o.order_id
group by p.product_id, p.product_name;

select
    c.customer_id,
    c.customer_name,
    count(o.order_id) as total_orders
from customer c
join orders o on c.customer_id = o.customer_id
where o.order_status <> 'cancelled'
group by c.customer_id, c.customer_name
having count(o.order_id) = (
    select max(order_count)
    from (
        select count(*) as order_count
        from orders
        where order_status <> 'cancelled'
        group by customer_id
    ) as customer_counts
);

select
    c.customer_id,
    c.customer_name,
```

```
    coalesce(sum(o.total_amount), 0) as total_spending
from customer c
left join orders o on c.customer_id = o.customer_id
    and o.order_status <> 'cancelled'
group by c.customer_id, c.customer_name
order by total_spending desc;

select round(avg(total_amount), 2) as average_order_value
from orders
where order_status <> 'cancelled';

select coalesce(sum(total_amount), 0) as total_sales
from orders
where order_status <> 'cancelled';

select * from customer;
select * from product;
select * from orders;
select * from order_details;
```